DocuMind – AI Technical Documentation Assistant

Generative AI Final Project · RAG + Prompt Engineering + Synthetic Data + Autonomous Research Agent

Prathamesh Shukla

April 2026

Contents

Project Overview

DocuMind is a production-quality, end-to-end Generative AI web application that helps developers generate, understand, and enhance technical documentation with inline citations, confidence scoring, and an autonomous research agent that grows its own knowledge base.

The system integrates **three required rubric components plus a substantive beyond-scope addition**:

1. **Prompt Engineering** – four role-separated system prompts with priority-ordered constraints, edge-case handling, and strict output contracts (Markdown for chat/docs, JSON for synthetic data and agent decisions).
2. **Retrieval-Augmented Generation (RAG)** – HTML scraping, semantic chunking, OpenAI embeddings, FAISS (cosine via L2-normalized inner product), top-k retrieval, and prompt injection with `[S#]` citations.
3. **Synthetic Data Generation** – diverse Q&A pair generation with category-forced diversity (factual / reasoning / edge-case / example), strict JSON schema, and a regex-fallback parser for defensive decoding.
4. **Bonus – Autonomous Research Agent** *(beyond scope)*: given only a natural-language topic, the agent plans diverse search queries, crawls DuckDuckGo, scrapes result pages, asks the LLM to judge each page's relevance, and auto-ingests the good ones. Progress is streamed to the UI live over Server-Sent Events.

Stack

- **Backend** – FastAPI, Pydantic v2, OpenAI SDK, FAISS, BeautifulSoup, Tenacity (retries).
- **Frontend** – React 18, Vite, Tailwind CSS, Framer Motion, React Router, react-markdown.
- **Tests** – pytest + FastAPI `TestClient`; 19/19 passing, mocked network for agent tests.

Demo

- **10-minute video demo**: https://youtu.be/HR--Zm4KlT0
- **GitHub repository**: https://github.com/psshukla02/documind
- **Runs offline without an API key** via deterministic stub mode – useful for graders without an OpenAI account.

This document compiles the rubric-alignment summary, architecture, implementation details, autonomous agent design, performance metrics, challenges encountered, ethical considerations, and future improvements.

Rubric Alignment

This project targets the *Generative AI Final Project* rubric. The rubric requires at least **two** of the five core components; DocuMind implements **three** deeply, plus a substantial beyond-scope addition.

Core Components – 3 of 5 Implemented

#	Component	Status	Primary Evidence
1	Prompt Engineering	**Done**	`backend/app/prompts/templates.py` – 4 role-separated system prompts, priority-ordered constraints, output contracts, edge-case fallbacks, regex-fallback parsers.
2	Fine-Tuning	–	*Not pursued; rubric floor is two components.*
3	RAG	**Done**	`backend/app/services/{scraper, chunker, vector_store, rag}.py` – scrape -> chunk -> embed -> FAISS -> retrieve -> compose -> cite.
4	Multimodal	–	*Not pursued; text-only by design.*
5	Synthetic Data	**Done**	`backend/app/services/rag.py::generate_synthetic` + `SYNTHETIC_SYSTEM` prompt; diversity-forced categories, strict JSON contract.

Beyond Scope – Autonomous Research Agent

This is a fully independent capability *not required by the rubric*:

- **Goal.** Turn a natural-language topic into a self-expanding knowledge base without human URL curation.
- **Location.** `backend/app/services/agent.py` + `backend/app/routers/agent.py` – `backend/app/services/search.py` + `frontend/src/pages/AgentPage.jsx`.
- **Capabilities:**
  - LLM-planned diverse search queries (`AGENT_PLANNER_SYSTEM`).
  - DuckDuckGo HTML scraping (no API key required).
  - Domain + extension blacklist (video, social, binary files).
  - LLM relevance judge per page (`RELEVANCE_JUDGE_SYSTEM`, strict-JSON decision).
  - Automatic chunking + embedding + storage.
  - De-duplication against the existing vector store.
  - Server-Sent Events live progress stream to the UI.

A single agent run on the topic *"FastAPI framework"* autonomously discovers the official FastAPI site and the FastAPI ReadTheDocs tutorial, judges them both as relevant, and ingests **105 chunks in 9.8 seconds** – no URL provided by the user.

Supporting Requirements

Rubric Item	Evidence
GitHub repo with source	https://github.com/psshukla02/documind – 75+ files, all deps pinned.

Rubric Item	Evidence
Setup instructions	`README.md` §Setup, `backend/run.sh`, `.env.example`.
Testing scripts	`tests/` – 5 pytest modules, 19 tests, 100% pass; offline `eval_retrieval.py`.
Example outputs	`README.md` §Example Outputs, `tests/sample_urls.json`.
Knowledge base	Runtime-persisted to `data/vector_store/` (FAISS index + JSON metadata).
Documentation PDF	This document.
10-minute demo video	https://youtu.be/HR--Zm4KlT0.
Web page	`website/index.html` (dark, polished landing page).
Evaluation metrics	`/api/metrics` endpoint + live dashboard (`frontend/src/pages/MetricsPage.jsx`).

Design Highlights

- **Hallucination mitigation** is built in, not bolted on: required inline citations, retrieval-score-weighted confidence bar, mandatory fallback phrase when context is thin.
- **Stub mode** ships with the application: if `OPENAI_API_KEY` is absent, embeddings are deterministic hash-seeded vectors and LLM outputs are templated – the full pipeline (search, retrieval, UI, metrics, SSE streaming) still runs end-to-end for graders without an API key.
- **Every answer is auditable.** Citations, source URLs, chunk scores, retrieval average, latency, and token count are surfaced per response.
- **Live observability.** A rolling in-memory metrics store powers a 5-second auto-refresh dashboard – counters, averages, and recent events.

System Architecture

DocuMind is a two-tier application: a FastAPI backend that owns all LLM, vector-store, and agent logic, and a React/Vite frontend that renders an Apple-styled dashboard and a live event stream for the research agent.

High-Level View

Diagram (Mermaid source below – render at https://mermaid.live/ to view).

```
flowchart LR
    U[User / Developer] --> UI[React UI<br/>Vite + Tailwind + Framer Motion]
    UI -->|HTTP/JSON /api/*| API[FastAPI<br/>CORS, routers, schemas]
    UI -->|SSE /agent/stream| API

    subgraph AGENT[Autonomous Research Agent]
        direction TB
        PLAN[Planner<br/>LLM: N diverse queries] --> SRCH[DuckDuckGo HTML search<br/>no API ke
        SRCH --> FLT[Filter<br/>blacklist + dedupe]
        FLT --> SCR[Scraper]
        SCR --> JUDGE[Relevance Judge<br/>LLM: keep or skip]
        JUDGE -->|keep| ING
    end

    subgraph RAG[RAG Pipeline]
        direction TB
        ING[Scraper<br/>requests + BS4] --> CHK[Chunker<br/>paragraph + heading aware]
        CHK --> EMB[Embedder<br/>OpenAI text-embedding-3-small]
        EMB --> VS[(FAISS<br/>IndexFlatIP + JSON meta)]
        VS --> RET[Retriever<br/>top-k cosine]
        RET --> PR[Prompt Composer<br/>system + user templates]
        PR --> LLM[OpenAI Chat<br/>gpt-4o-mini]
    end

    API --> AGENT
    API --> RAG
    API --> MET[Metrics Store<br/>in-memory rolling window]

    LLM --> API
    MET --> API
    API --> UI
```

Chat Request Sequence

Diagram (Mermaid source below – render at https://mermaid.live/ to view).

```
sequenceDiagram
    participant U as User
    participant F as Frontend
```

```
    participant B as FastAPI
    participant E as Embeddings API
    participant V as FAISS
    participant L as Chat LLM

    U->>F: Types question, presses Send
    F->>B: POST /api/chat {query}
    B->>E: embed(query)
    E-->>B: 1536-dim vector
    B->>V: search(normalized vec, k=4)
    V-->>B: top-k chunks + cosine scores
    B->>B: build_chat_user_prompt(query, chunks)
    B->>L: chat(system=CHAT_SYSTEM, user=prompt, temp=0.2)
    L-->>B: answer text + token usage
    B->>B: compute confidence + record metrics
    B-->>F: JSON {answer, citations, confidence, latency, tokens}
    F-->>U: Markdown render + citation pills
```

Ingestion Sequence

Diagram (Mermaid source below – render at https://mermaid.live/ to view).

```
sequenceDiagram
    participant U as User
    participant F as Frontend
    participant B as FastAPI
    participant S as Scraper
    participant C as Chunker
    participant E as Embeddings API
    participant V as FAISS

    U->>F: Submits URL
    F->>B: POST /api/ingest {url}
    B->>S: scrape_url(url)
    S-->>B: ScrapedPage(title, text)
    B->>C: chunk_text(text, size=800, overlap=120)
    C-->>B: list[Chunk]
    B->>E: embed(chunks)  -- batched
    E-->>B: list[vector]
    B->>V: add(doc_id, vectors, metadata)
    V-->>B: acknowledgement
    B-->>F: {doc_id, title, chunks, latency_ms}
```

Agent Sequence (Beyond Scope)

Diagram (Mermaid source below – render at https://mermaid.live/ to view).

```
sequenceDiagram
    participant F as Frontend (EventSource)
```

```
participant B as FastAPI /agent/stream
participant P as Planner LLM
participant D as DuckDuckGo
participant S as Scraper
participant J as Judge LLM
participant R as RAG ingest

F->>B: GET /api/agent/stream?topic=...
B->>P: plan N diverse queries
P-->>B: {"queries": [...]}
B-->>F: SSE: {type: plan, queries}
loop per query
    B->>D: search(query)
    D-->>B: result list
    B-->>F: SSE: {type: search_results}
    loop per candidate (dedup + blacklist filtered)
        B->>S: scrape(url)
        S-->>B: page
        B-->>F: SSE: {type: scrape_done}
        B->>J: judge(topic, page)
        J-->>B: {decision, reason}
        B-->>F: SSE: {type: judge}
        alt keep
            B->>R: chunk + embed + add
            B-->>F: SSE: {type: ingest}
        end
    end
end
B-->>F: SSE: {type: done, summary}
```

Module Responsibilities

Module	Responsibility
`app/core/config.py`	Env-loaded settings via `pydantic-settings`
`app/core/metrics.py`	Thread-safe rolling metrics window (200 events) + counters
`app/core/logging.py`	Minimal structured logger
`app/services/llm.py`	OpenAI wrapper with tenacity retries + stub-mode fallback
`app/services/search.py`	DuckDuckGo HTML search, link unwrap, blacklist filter
`app/services/agent.py`	Autonomous research loop (plan -> search -> scrape -> judge -> ingest)
`app/services/scraper.py`	Fetch + clean HTML (strips scripts, nav, footer, ads, etc.)
`app/services/chunker.py`	Paragraph + heading-aware semantic chunking with tail overlap
`app/services/vector_store.py`	FAISS index + JSON metadata, disk-persisted, thread-safe add
`app/services/rag.py`	End-to-end RAG: ingest, chat, generate-docs, synthetic
`app/prompts/templates.py`	4 system prompts and their user-side builders
`app/routers/*`	Thin FastAPI route handlers, validated via `schemas.py`
`frontend/src/pages/*`	Home, Chat, Agent, KB, DocsGen, Synthetic, Metrics

Module	Responsibility
`frontend/src/components/*`	Reusable UI primitives (Card, Button, Input, Markdown, Hero…)

Deployment Topology

Local-first. Both services run on the developer's machine:

- Backend on port **8000** (Uvicorn with hot reload).
- Frontend on port **5173** (Vite dev server). Vite's `server.proxy` forwards `/api/*` to the backend, so the React bundle makes same-origin requests – no CORS gymnastics in development.

For production: any WSGI/ASGI host (uvicorn behind Nginx, AWS App Runner, Fly, Render), a persistent volume mounted at `data/vector_store/`, and the React bundle served statically.

Request Lifetimes and Timeouts

Path	Typical	Notes
`GET /api/health`	<10 ms	Pure in-process check
`GET /api/metrics`	<10 ms	Snapshot of the rolling window
`POST /api/chat`	1-3 s	1 embed call + 1 chat call + prompt composition
`POST /api/ingest`	1-3 s	HTTP fetch + parse + chunk + batch embed + index append
`POST /api/generate-docs`	3-5 s	Longer output (~1400 max tokens)
`POST /api/synthetic-data`	4-7 s	Full-document source, temperature 0.6, JSON contract
`GET /api/agent/stream`	10-60 s	Bounded by `num_queries × per_query` scrape+judge steps

Per-network timeout is **30 s** (scraper + LLM API), configurable via the `REQUEST_TIMEOUT` env var. OpenAI calls use `tenacity` with exponential backoff, up to three retries.

Implementation Details

This section breaks down each of the three rubric-aligned components – RAG, Prompt Engineering, Synthetic Data – along with the confidence scoring, observability, and fault-tolerance infrastructure that glues them together into a production-quality application.

1. RAG Pipeline

1.1 Ingestion

`app/services/scraper.py` fetches a URL with a browser-like User-Agent (`Mozilla/5.0 (compatible; AI-Doc-Assistant/1.0)`) and a 30-second timeout, then parses with BeautifulSoup (lxml parser). It strips the following elements from the tree before extracting text:

`script, style, noscript, iframe, svg, nav, footer, aside, form`

The remaining `<main>` / `<article>` / `<body>` text is flattened and whitespace-normalized (collapsing runs of blank lines and multi-space runs). Pages with fewer than 40 characters of extracted content are rejected with a `ValueError("No meaningful content extracted")` – catches 404s served as 200s, login walls, and heavy SPA pages that return a shell HTML.

1.2 Chunking

`app/services/chunker.py` implements what we call "semantic-ish" chunking: heading- and paragraph-aware without a sentence transformer in the hot path.

1. Split on blank-line runs into paragraphs.
2. Pack paragraphs into windows of approximately `chunk_size` characters (default 800).
3. **Heading-aware break**: a paragraph matching `r"^(#{1,6}\s|\d+\.\s|[A-Z][A-Za-z0-9]{0,60}:\s*$)"` forces a chunk boundary if the current buffer is already at least half full. Keeps semantic units (sections) together.
4. **Oversized paragraph handling**: paragraphs longer than `chunk_size` are sliced into overlapping windows directly.
5. **Tail overlap**: each chunk gets the last `chunk_overlap` characters (default 120) of the previous chunk prepended, so retrieval does not miss boundary-spanning facts.

Defaults chosen empirically to balance retrieval granularity against embedding cost.

1.3 Embedding

OpenAI's `text-embedding-3-small` (1536 dimensions). In stub mode we fall back to a deterministic SHA-256-seeded pseudo-random vector (384 dimensions) – useless for semantics but keeps every downstream code path exercised so graders without an API key see the full application work end-to-end.

1.4 Vector Store

`faiss.IndexFlatIP` over **L2-normalized** vectors, which makes inner product equal to cosine similarity. Metadata (chunk text, source URL, title, ordinal) lives in a parallel `meta.json` keyed by FAISS row index. Both files persist under `data/vector_store/` and auto-load at startup.

Design choice: `IndexFlatIP` (exact search) instead of an approximate index (HNSW / IVF). At the scale of a student project (a few thousand chunks) the latency difference is negligible and the exact guarantee simplifies evaluation.

1.5 Retrieval

1. Embed the query with the same model used for chunks.
2. L2-normalize.
3. `index.search(q, k)` returns top-k cosine similarities and row indices.
4. Look up metadata for each hit.
5. Surface scores back through the API so the UI can colour-grade the confidence bar.

1.6 Prompt Injection

The top-k chunks are labelled `[S1]`, `[S2]`, … and inlined into the user-side prompt. The chat system prompt requires the model to cite those labels; the frontend rewrites matched labels into interactive pills that link to the source URL.

2. Prompt Engineering

`app/prompts/templates.py` is the single source of truth. Three design patterns are used consistently across all four system prompts.

2.1 Role Separation

System messages carry **identity + hard constraints**. User messages carry **task + data + output contract**. The split is not cosmetic – OpenAI's chat endpoint weights system and user roles differently, and the split keeps invariant rules visible across multi-turn conversations (future work).

2.2 Instruction Hierarchy

Every system prompt lists its constraints in **priority order**:

> Hard constraints (in priority order): 1. Ground every factual claim in the provided sources. 2. If the sources do not contain enough information to answer, say so explicitly. Do NOT invent APIs, flags, function names, or version numbers. 3. …

When constraints could conflict – "be helpful" vs. "don't hallucinate" – the model has an explicit tiebreaker. This dramatically reduces the "but the user asked for code" failure mode we observed during development.

2.3 Output Contracts

Each prompt specifies the exact shape of the expected response, which drastically reduces parsing failures:

- **Chat**: Markdown with inline `[S#]` citations + a Sources section.
- **Doc gen**: strict section order – Overview, Quick Start, Usage, Parameters, Edge Cases, Related – every section with an `H2` heading.

- **Synthetic**: strict JSON schema with a top-level `"pairs": [...]` array. The prompt explicitly says "Return ONLY the JSON object."
- **Agent planner**: strict JSON `{"queries": [...]}`, <=12-word queries.
- **Relevance judge**: strict JSON `{"decision": "keep"|"skip", "reason": "..."}`.

2.4 Edge-Case Handling Inline

Instead of wiring up a separate "fallback" prompt, the main prompt tells the model what to do when the input is degenerate:

> If the context is empty or insufficient, say exactly: *"I don't have enough information in the knowledge base to answer this confidently."* and suggest what to ingest.

This reduces hallucination without a separate guardrail model; we can then filter by substring in post-processing if needed.

2.5 Prompt Taxonomy

Prompt	System role	User data	Output contract
`CHAT_SYSTEM`	Senior technical doc assistant	query + ranked chunks	Markdown + `[S#]` + sources
`DOC_GEN_SYSTEM`	Senior technical writer	topic + optional code	6-section Markdown
`SYNTHETIC_SYSTEM`	Synthetic data generator	full-doc source	strict JSON pairs
`AGENT_PLANNER_SYSTEM`	Research query planner	topic + N	strict JSON queries
`RELEVANCE_JUDGE_SYSTEM`	Relevance judge	topic + page excerpt	strict JSON decision

3. Synthetic Data Generation

`rag.generate_synthetic()` bundles chunks from a target document (capped at 6000 characters to keep costs bounded), injects them into `SYNTHETIC_SYSTEM` plus `build_synthetic_user_prompt()`, and parses the JSON response.

The prompt enforces diversity by requiring **categorical coverage**:

> Diversity Requirements: - At least one factual lookup question. - At least one "why/how" reasoning question. - At least one edge-case or pitfall question. - Vary question length and phrasing.

Categories: `factual | reasoning | edge_case | example`. Difficulty: `easy | medium | hard`.

Uses for the generated pairs

1. **Inline examples** in the UI (Synthetic Data page) with colour-coded category pills.
2. **Prompt improvement** – pairs can be exported via "Copy JSON" and re-used as few-shot examples in future versions of the chat prompt.
3. **Evaluation** – the Q&A pairs are a ready-made regression test set for the retriever (do the answers still surface the right chunks?).

Defensive JSON parsing

The ideal case: `json.loads(response_text)`. Reality: LLMs sometimes wrap JSON in prose or fenced blocks. We handle three layers:

1. Try `json.loads` on the raw text.
2. On failure, extract the first `{...}` block via regex `r"\{.*\}"` with `re.DOTALL` and retry.
3. If both fail, return an empty list and log the first 200 characters of the raw response. The UI displays "The model returned no usable pairs" rather than a 500.

4. Confidence Scoring

Confidence is a cheap proxy, not a calibrated probability:

```
confidence = 0.6 × top_retrieval_score + 0.4 × citation_coverage
citation_coverage = cited_sources / total_retrieved_sources
```

- **Low top score** -> retriever did not find relevant material.
- **Low citation coverage** -> model did not ground in what it got, suggesting hallucination risk.

The UI renders this as a colour-graded bar: mint >=70%, peach 40-70%, rose <40%.

5. Observability

`app/core/metrics.py` is a thread-safe in-memory store with a rolling 200-event window plus integer counters. Every chat / ingest / docs / synthetic / agent call records `{kind, latency_ms, retrieval_score, tokens, ...}`.

`GET /api/metrics` returns:

- Counters (`chat_requests`, `ingest_requests`, `errors`, `agent_requests`, …).
- Rolling averages (latency, retrieval score, tokens, chunks/doc).
- Recent 20 events for the live feed.

Swap for Redis or Prometheus in production – the interface is already narrow (two methods: `record`, `bump`).

6. Frontend

React 18 + Vite for fast HMR. Tailwind CSS for design consistency. Framer Motion for subtle micro-interactions (page transitions, button hover scale, bubble fade-ins, staged event rows).

Seven pages:

- **Home** – pastel hero, feature grid, CTA.
- **Chat** – rounded bubble conversation with confidence bar, citation pills, copy buttons, typing indicator, sticky glass input bar.
- **Research Agent** – topic input, live SSE event timeline, summary stat cards.
- **Knowledge Base** – URL / text ingestion, document table, clear-all.
- **Doc Generator** – topic + code input, rendered Markdown output.
- **Synthetic Data** – document picker, category-colored pair cards, Copy JSON.
- **Metrics** – stat cards + rolling event table, 5-second auto-refresh.

A thin `api/client.js` centralizes `fetch` with error handling.

7. Error Handling

- **Empty input** – validated by Pydantic at the route layer (returns 422 with field-level detail).
- **No relevant docs** – model is instructed to output the canonical "I don't have enough information" string; confidence drops.
- **OpenAI failures** – `tenacity` retries with exponential backoff (3 tries, 1-8 s jitter).
- **Missing API key** – stub mode kicks in transparently; sidebar shows a warning chip.
- **Scrape failures** – 400 with the upstream HTTP error message.
- **SSE dropped** – the frontend's `EventSource.onerror` surfaces the error to the user and closes the stream cleanly.
- **Frontend** – every page shows loading, error, and empty states.

8. Testing

19 tests in 5 modules (`tests/`):

- `test_health.py` – root + health endpoints
- `test_chunker.py` – paragraph merging, heading-aware breaks, oversized paragraph handling
- `test_ingest_and_chat.py` – text ingestion, chat with retrieval, empty-query rejection, KB reset
- `test_synthetic_and_docs.py` – doc generation, synthetic data, metrics counters
- `test_agent.py` – mocked DDG + scraper, full research loop, de-duplication, blacklist, SSE streaming, scrape-failure handling

All tests run in stub mode against a temp vector store so a developer's real index is never touched.

Autonomous Research Agent

This is the project's **beyond-scope** capability. The rubric requires *at least two* of five Gen AI components; DocuMind implements three (Prompt Engineering, RAG, Synthetic Data) and then takes a further step the rubric does not ask for: an agent that turns a natural-language topic into a self-expanding knowledge base with **no URLs from the user**.

Motivation

RAG systems are only as good as their knowledge base, and curating a knowledge base by hand is tedious. The research agent automates curation – the user types a topic, the agent decides what to read, the knowledge base grows in minutes.

The Loop

Diagram (Mermaid source below – render at https://mermaid.live/ to view).

```
flowchart TD
    A[User submits topic] --> B[LLM: plan N diverse queries]
    B --> C{For each query}
    C --> D[DuckDuckGo HTML search]
    D --> E[Dedup + blacklist filter]
    E --> F[Scrape each candidate URL]
    F --> G[LLM: relevance judge<br/>keep / skip]
    G -->|keep| H[Chunk + embed + store<br/>FAISS]
    G -->|skip| I[Log skip reason]
    H --> C
    I --> C
    C --> J[Emit summary event]
```

Components

File	Role
`app/services/search.py`	DuckDuckGo HTML search; resolves wrapped links
`app/services/agent.py`	Orchestration loop; yields progress events
`app/prompts/templates.py`	`AGENT_PLANNER_SYSTEM`, `RELEVANCE_JUDGE_SYSTEM`
`app/routers/agent.py`	`POST /agent/research`, `GET /agent/stream` (SSE)
`frontend/src/pages/AgentPage.jsx`	Live event timeline, summary card, topic input

Search

We **do not require an API key**. The agent POSTs to `https://html.duckduckgo.com/html/`, which returns a static HTML result list, and parses it with BeautifulSoup. Outbound links are wrapped in `/l/?uddg=<url>` and we unwrap them via `urllib.parse`.

Blacklist

Domains that either block bots or lack text content are skipped at the search step:

- Video: `youtube.com`, `vimeo.com`, `tiktok.com`
- Social: `x.com`, `twitter.com`, `facebook.com`, `instagram.com`, `linkedin.com`, `pinterest.com`, `reddit.com`
- Binary extensions: `.pdf`, `.zip`, `.doc(x)`, `.ppt(x)`, `.xls(x)`, `.mp4`, `.mp3`, `.mov`, `.avi`

`is_blacklisted()` checks host lowercase match and extension regex. Covered by a dedicated test (`test_search_url_unwrap_and_blacklist_helpers`).

Planning

`AGENT_PLANNER_SYSTEM` asks the LLM to produce **N diverse search queries** with priority-ordered constraints:

1. Queries must be diverse (different angles, not paraphrases).
2. Prefer queries likely to hit authoritative docs.
3. Each query <=12 words.
4. Output strict JSON `{"queries": ["...", "..."]}`.

A regex-fallback parser recovers the JSON if the model adds stray prose.

In stub mode the planner returns deterministic variants (`{topic}`, `{topic} tutorial`, `{topic} official documentation`, …), so the downstream search/scrape still exercises real network code.

Relevance Judging

For each scraped page we call the LLM with `RELEVANCE_JUDGE_SYSTEM`, which enforces a strict JSON contract:

```
{"decision": "keep" | "skip", "reason": "one short sentence"}
```

The judge sees the topic, page title, search snippet, and the first 1500 characters of the cleaned body – enough signal, cheap on tokens. A conservative default (skip) is applied if the JSON can't be parsed at all.

Criteria for "keep":

- Directly relevant to the topic.
- Contains substantive technical content.
- In English.
- Not a login wall, 404, cookie banner, or ad-heavy landing page.

Deduplication

Before scraping, the agent pulls the existing knowledge-base URLs from `VectorStore.list_documents()` and tracks them in a `seen` set. A second run on the same topic produces `skip` events rather than re-ingesting pages. This is explicitly covered by a test (`test_agent_deduplicates_existing_sources`).

Streaming Protocol

`GET /api/agent/stream` emits Server-Sent Events (one JSON object per `data:` line):

Event	Payload
`start`	`{topic, num_queries, per_query}`
`plan`	`{queries: [...]}`
`search_start`	`{iter, query}`
`search_results`	`{query, urls: [{url, title, snippet}, ...]}`
`scrape_start`	`{url, title}`
`scrape_done`	`{url, chars, title}`
`scrape_failed`	`{url, reason}`
`judge`	`{url, decision, reason}`
`ingest`	`{url, title, chunks, doc_id}`
`ingest_failed`	`{url, reason}`
`skip`	`{url, reason}`
`done`	`{summary: {topic, queries, scraped, ingested, skipped, ...}}`
`error`	`{message}`

The React page consumes this with `EventSource`.

Example Run (Real, No API Key)

Topic: `"FastAPI framework"`, 1 query, 2 results per query.

The agent autonomously:

1. **Planned** the query `"FastAPI official documentation"`.
2. **Searched** DuckDuckGo -> 4 results.
3. **Scraped** `https://fastapi.tiangolo.com/` (13,734 chars).
4. **Judged** as keep: *"Directly relevant and contains substantive technical content about FastAPI."*
5. **Ingested** 21 chunks.
6. **Scraped** `https://fastapi-tutorial.readthedocs.io/en/latest/` (46,859 chars).
7. **Judged** as keep.
8. **Ingested** 84 chunks.
9. **Done**: 2 documents, **105 chunks, 9.8 seconds**, zero URLs provided by the user.

Operational Limits

- Max 5 queries per run × 5 results per query (hard-coded in `ResearchConfig`; enforced at the Pydantic layer).
- Per-page scrape timeout: `REQUEST_TIMEOUT` (default 30 s).
- Per-scrape character cap for the judge prompt: 1500 chars.
- No recursion / link-following: one hop only, one topic per run. Keeps it simple and auditable.

Why This Matters (for the Rubric)

The agent demonstrates:

- **Technical innovation**: combines prompt engineering, planning, search, scraping, and RAG ingestion into a single coherent workflow.
- **Creative application**: traditional RAG requires manual KB curation – this removes that friction.

- **Real-time UX**: SSE streaming gives the user a live, auditable picture of every decision the LLM makes.
- **Robustness**: dedup, blacklist, scrape failure handling, conservative defaults on JSON parse errors.

Safety Notes

- The scraper identifies itself with a browser User-Agent and respects HTTP timeouts. It does **not** honour robots.txt – do not point it at sites you do not own or have explicit authorization to crawl.
- Every ingested page has its source URL stored alongside it, so every answer that cites the page can link back to the original.
- The blacklist and relevance judge combine to keep anti-bot domains and low-quality pages out. They are not perfect; audit the knowledge base periodically with the "Clear All" button.

Performance Metrics

Measurements below are representative from a local MacBook Pro (M-series, 16 GB), against `gpt-4o-mini` and `text-embedding-3-small`. Numbers vary by network. Bench harness: manual wall-clock via `time.time()` in `rag.py`, exposed via `/api/metrics`.

Latency (End-to-End)

Operation	p50	p95	Notes
`POST /ingest` (URL)	1.8 s	3.2 s	Dominated by HTTP fetch (300-800 ms) and embed call (800-2000 ms).
`POST /chat`	1.4 s	2.6 s	1 embed + 1 chat call.
`POST /generate-docs`	3.2 s	5.1 s	Longer output (up to 1400 tokens), ~2-4× chat latency.
`POST /synthetic-data`	4.0 s	6.5 s	Larger context, temperature 0.6 for diversity.
`GET /agent/stream`	10-60 s	–	Bounded by `num_queries × per_query`. One real run: 2 queries × 3 URLs = 45 s.
`GET /metrics`	< 10 ms	< 20 ms	In-memory snapshot.

Stub mode cuts every LLM/embed call to < 5 ms, so the full pipeline runs in tens of milliseconds end-to-end – useful for UI regression testing.

Latency Budget (Chat)

A typical chat call decomposed:

Stage	Share	Typical
Embed query (`text-embedding-3-small`, 1 token)	~25 %	350 ms
FAISS search (`IndexFlatIP`, ~1k vectors, k=4)	<1 %	<5 ms
Prompt composition + format	<1 %	<5 ms
Chat completion (`gpt-4o-mini`, ~600 out tokens)	~74 %	1050 ms
Serialization + response	<1 %	<10 ms

Implication. The retrieval step is not the bottleneck at this scale; LLM latency is. Swapping the retriever for something more sophisticated (BM25 hybrid, cross-encoder re-ranker) would barely move the p50.

Retrieval Quality

`/api/metrics` surfaces mean cosine similarity per chat request. Observed ranges on a KB seeded with `https://fastapi.tiangolo.com/` plus a Pydantic documentation page:

Query type	Typical top-1 score
On-topic (e.g. "What is FastAPI?")	0.70 - 0.82
Adjacent (e.g. "async Python basics")	0.45 - 0.60
Off-topic (e.g. "capital of Iceland")	0.15 - 0.30

The retriever's top-1 score, combined with citation coverage, feeds the **confidence bar** shown in the UI:

```
confidence = 0.6 × top_score + 0.4 × citations_used / retrieved
```

In practice the bar reads 65-85 % for on-topic queries, 30-50 % for adjacent ones, and <25 % for off-topic – a useful forcing function for users to re-read the answer before trusting it.

Token Usage and Cost

Typical chat call: ~1200 input tokens + 300-600 output tokens on `gpt-4o-mini`.

At the January-2026 `gpt-4o-mini` price sheet ($0.15 per 1M input, $0.60 per 1M output), that is approximately **$0.0005 per chat** and **$0.02 per agent run** (planner + per-page judges + ingestion is embedding-only). Verify before production use.

Embedding cost is negligible: `text-embedding-3-small` at $0.02 per 1M tokens, and each chunk is ~200 tokens, so embedding a ~100-chunk document costs a fraction of a cent.

Throughput

FastAPI + Uvicorn in a single worker handles ~**40 concurrent chat requests per second** when LLM calls are mocked (stub mode). Real throughput is bounded by OpenAI rate limits, not the local server.

Test Suite Performance

All 19 pytest tests complete in **under 1 second** on stub mode (0.77 s wall-clock on the reference machine) – fast enough to run on every save during development.

Observability

The **Metrics** page auto-refreshes every 5 seconds and surfaces:

- Request counters per endpoint
- Rolling average latency per endpoint
- Rolling average retrieval score
- Token usage average
- 20 most recent events with timestamps

This is what you would lift into Prometheus/Grafana in a production deployment – the interface is intentionally narrow and Prometheus- compatible.

Challenges and Solutions

Every non-trivial decision involved a trade-off. This section documents the ones that shaped the final architecture.

1. Semantic-enough chunking without a heavy framework

Challenge. Naive fixed-size chunking (every 1000 chars) splits mid-sentence and loses coherence. Full semantic chunking via a sentence transformer is overkill for a student project – adds a large dependency and a warm-up cost for every startup.

Solution. Paragraph-first packing with heading-aware boundary breaks and a tail overlap. Chunks are coherent paragraphs or groups of paragraphs, never half-sentences. The overlap guarantees boundary-spanning facts are retrievable.

Result. On-topic retrieval scores consistently in the 0.7-0.8 range, competitive with sentence-transformer chunkers we benchmarked informally, at zero marginal cost.

2. Hallucination reduction

Challenge. Even with relevant context, LLMs will invent API names and version numbers when the sources do not quite answer the question.

Solution. Three layers:

- **Priority-ordered constraints** in the system prompt.
- An **explicit fallback phrase** the model is told to output verbatim when context is insufficient. This makes hallucinations detectable by a simple substring filter.
- **Citation-coverage scoring** – answers that fail to cite `[S#]` tags are surfaced to the user via a low confidence bar.

Result. Qualitatively, the model reliably emits the canonical fallback phrase on off-topic queries we tested. The confidence bar drops below 25 % for those cases, which is visible before the user reads the answer.

3. Stub-mode parity

Challenge. We wanted the app to be fully demoable without an API key, but the frontend should not have to know the backend is stubbed.

Solution. The stub path is hidden inside `LLMClient`. It returns the same shape (`text`, `tokens`, `model`) as a real OpenAI response. Embeddings use hash-seeded pseudo-random vectors of the same dimension, so FAISS is happy. A single `stub_mode` flag surfaces in `/api/health` so the UI can show a warning chip.

Result. Graders without an OpenAI key can still exercise the full agent loop, ingestion, retrieval, and UI flows – only the LLM text output is templated.

4. Forcing JSON out of a chat model

Challenge. `gpt-4o-mini` sometimes wraps JSON output with prose or Markdown fences, breaking `json.loads`. The synthetic-data endpoint and both agent prompts need strict JSON.

Solution. A regex fallback extracts the first `{...}` block from the response body if the direct parse fails. If both fail, we return a conservative default (empty pair list, `"decision": "skip"`) rather than a 500.

Result. Zero observed 500s from malformed JSON in hundreds of test runs.

5. CORS and dev ergonomics

Challenge. React (port 5173) and FastAPI (port 8000) run on different origins. We wanted hot reload without re-CORS-ing every endpoint during iteration.

Solution. Vite's `server.proxy` routes `/api/*` to the backend during development, so the frontend code makes same-origin requests. In production, `CORS_ORIGINS` env var controls the allowlist.

6. FAISS dimension detection at startup

Challenge. We did not want to hardcode the embedding dimension – it differs between stub mode (384), `text-embedding-3-small` (1536), and `text-embedding-3-large` (3072).

Solution. On first access, `get_vector_store()` either uses the stub constant or fires a one-token probe embedding to measure the dim, then caches the dimension for the lifetime of the process. The same detection lets us rebuild the store cleanly if the user switches models.

7. SSE streaming without breaking the proxy

Challenge. Server-Sent Events need un-buffered text to work smoothly, but dev proxies and browsers will sometimes buffer.

Solution. Explicit response headers: `Cache-Control: no-cache, no-transform`, `X-Accel-Buffering: no`, `Connection: keep-alive`. Vite's proxy passes through cleanly once these are set.

8. Preventing duplicate ingestion from the agent

Challenge. Run the agent on the same topic twice and it would re-ingest the same pages, polluting retrieval with duplicates and inflating chunk counts.

Solution. `_existing_sources()` pulls URLs from the vector store and seeds the agent's `seen` set before the loop starts. Duplicates produce a `skip` event rather than a re-ingest.

Result. Idempotent agent runs, verified by `test_agent_deduplicates_existing_sources`.

9. Rendering Mermaid diagrams in the PDF

Challenge. Pandoc + xelatex cannot render Mermaid out of the box. We did not want to drop the diagrams silently, and pulling in `mermaid-cli` adds a heavy Node dependency just for the PDF build.

Solution. The `build_pdf.py` preprocessor converts Mermaid fences into a short note + the raw source inside a verbatim block. Readers see a caption pointing them at https://mermaid.live to view the rendered version. The Markdown versions in the repo render Mermaid natively in GitHub and most Markdown viewers.

10. Apple-style UI without over-designing

Challenge. The starting UI was a dark dashboard. The user wanted Apple-style pastel light theme – but we had to keep every component functional and not regress UX.

Solution. A focused refactor: new Tailwind palette (brand / mint / lavender / peach / ink), Framer Motion for micro-interactions only, glassmorphism sidebar, re-written Card and Button primitives that stay functionally identical. The pages changed visually but behaviourally are the same – all 19 backend tests still pass without modification.

Result. Visual overhaul in one commit, zero regressions, fast Vite builds (~1.5 s), bundle size under 150 KB gzipped.

Ethical Considerations

Bias

- **Source bias.** The assistant is only as representative as the documents ingested. If a team only ingests blog posts from one vendor, answers will reflect that vendor's opinions. We surface the source URL on every citation so the user can audit the provenance.
- **Model bias.** `gpt-4o-mini` inherits whatever biases exist in OpenAI's training data. Our system prompts minimize free-form opinions – *"Hard constraint: ground every factual claim in the provided sources"* – but cannot eliminate them.
- **Retrieval bias.** Top-k retrieval will over-represent pages with dense, keyword-matching prose. Blog posts with lots of signposting can beat better-but-sparser reference docs. Mitigation: the UI shows *all* retrieved source titles and scores, so the user sees the full set, not just what was quoted.

Hallucination

- We never rely on the model to be honest by default. The prompt contract requires inline citations; the UI highlights missing citations via the confidence bar; the retrieval-score display makes it obvious when the model is answering from thin context.
- Even so, the system **can** still produce plausible-sounding wrong answers. Users must treat every output as a draft to be verified, not ground truth. The README and chat empty-state frame the tool that way explicitly.

Data Privacy

- Ingested content is stored locally in `data/vector_store/`. Nothing is exfiltrated except the prompts sent to OpenAI at query time.
- **Do not ingest proprietary or PII-containing documents without understanding OpenAI's data-retention policy** (API traffic is retained for abuse monitoring for 30 days by default; zero-retention agreements are available on enterprise plans).
- The scraper uses a bot-identifying User-Agent and a 30-second timeout. It does not attempt to bypass paywalls or authentication. It does not crawl recursively – one URL per ingest, or one topic per agent run.

Misuse

Risks and mitigations:

Risk	Mitigation
Generating fake docs to impersonate a library	Citations and confidence scoring make unverified outputs obvious.
Mass scraping of third-party sites	No built-in crawler – single-URL ingestion only, agent is one-hop.
Prompt injection via scraped content	System prompt runs first; user-visible citations expose injected content.
Leaking API keys in logs	Logger formats never include env vars; `.env` is gitignored.

Risk	Mitigation
Knowledge-base pollution from bad sources	Relevance judge + blacklist + dedup + admin "Clear All" button.

Intellectual Property

- Ingested text is stored verbatim in the vector store's metadata. The UI surfaces the source URL next to every quote, so attribution is preserved end-to-end.
- Generated outputs will paraphrase ingested text. This is a derivative use that under fair-use doctrine is generally acceptable for research, education, and internal engineering, but it is **not** automatically cleared for redistribution. Users should check the license of the original source before publishing generated docs.

Responsible Use Checklist

- [OK] Do ingest public documentation, your team's internal wikis, or source code you own.
- [OK] Do treat generated docs as drafts requiring human review.
- [OK] Do clear the knowledge base when switching projects – stale context leaks across domains.

☒ Don't ingest medical, legal, or financial documents and rely on the output without a qualified professional.
☒ Don't strip citations before sharing generated content – they're the user's audit trail.
☒ Don't point the agent at sites whose terms of service prohibit automated access.

Environmental Considerations

LLM inference has a non-trivial energy cost. Mitigations baked in:

- `gpt-4o-mini` (smaller model) as default, not `gpt-4o`.
- `temperature=0.2` for chat, minimizing regeneration cycles.
- Token caps on every prompt (`max_tokens` 900 / 1200 / 1400) to prevent runaway outputs.
- Stub mode for development and grading so contributors can test without burning real compute.

Future Improvements

Retrieval

- **Hybrid search.** Combine dense (embeddings) with sparse (BM25) for better recall on proper nouns and rare tokens. `rank_bm25` adds ~10 lines of code and one dependency.
- **Query rewriting.** A cheap LLM rewrite step ("expand abbreviations, split compound questions") can improve top-k hit rate without changing the retriever.
- **Cross-encoder re-ranking.** After the FAISS top-20, re-rank with a cross-encoder (e.g. `bge-reranker-base`) before feeding top-k to the LLM. Expected gain: +5-10 points on retrieval precision.
- **Per-document deletion.** Currently `DELETE /knowledge-base` clears everything; add a `DELETE /knowledge-base/{doc_id}` route and corresponding UI action.
- **Chunk-level deletion** so failed ingests can be rolled back.

Prompt Engineering

- **Few-shot from synthetic data.** Export high-quality synthetic Q&A pairs and inject the top 2-3 as few-shot examples in the chat system prompt. The infrastructure for this already exists – the Synthetic page has a Copy JSON button for the same reason.
- **Self-consistency.** For high-stakes queries, call the LLM 3× with temperature 0.7 and pick the answer with the highest citation coverage.
- **Structured output.** Migrate doc generation to OpenAI's response-format JSON mode so the UI can render richer, sectioned documentation with guaranteed schema.
- **Conversation memory.** Chat currently sends a single query. A small conversation-history window would let follow-ups work naturally.

Research Agent

- **Multi-hop research.** Let the agent follow interesting links from an ingested page, bounded by a budget. Current agent is one-hop.
- **Iterative refinement.** After the first pass, ask the LLM which sub-topics are still weak and plan a second round of queries.
- **Quality filter on ingestion.** Reject documents whose average retrieval score on their own chunks is low (suggests incoherent content).
- **Source diversity.** Enforce a rule that at most two URLs per host are ingested per run, preventing one site from dominating.

UX

- **Streaming chat responses.** Swap `/api/chat` for an SSE endpoint. The agent page already has the client-side plumbing for this.
- **Document explorer.** Click a citation in chat to jump to that chunk in the Knowledge Base viewer.
- **Saved sessions.** Persist conversation history in `localStorage` so refreshes do not lose context.
- **Mobile polish.** The layout stacks reasonably on mobile, but the sidebar should collapse into a hamburger on narrow screens.

Evaluation

- **Golden-set harness.** Ship N synthetic Q&A pairs per ingested doc, measure retrieval hit@k and answer-contains-expected on every PR.
- **Track evaluation scores over time.** Extend the Metrics page with a time-series chart.
- **A/B on prompt variants.** Small `prompts/variants/*.py` directory
  - a `PROMPT_VARIANT` env var would let us compare.

Productionization

- **Auth.** Currently open; add a simple API-key guard or OAuth proxy.
- **Persistent vector store.** Swap FAISS on-disk for Qdrant / pgvector for multi-process deployments.
- **Docker Compose.** Single `docker compose up` to run backend + frontend.
- **Observability.** Replace in-memory metrics with Prometheus + Grafana.
- **CI.** GitHub Actions workflow running pytest + `npm run build` on every push.
- **Rate limiting.** Per-IP token bucket on `/api/*` to prevent abuse if the app is ever exposed publicly.